

Guía de desarrollo — Plantilla MVC

Guía práctica y **genérica** para trabajar sobre cualquier proyecto generado con `New-MvcProject.ps1` (proyecto/portal principal) y `Add-MvcModule.ps1` (módulos enlazados).

Stack: **ASP.NET MVC 5 · .NET Framework 4.8 · Entity Framework 6 (Database-First) · IIS Express + SQL Server** .

En los ejemplos se usa el proyecto `MiApp` y el módulo `Inventario` . Reemplázalos por los nombres reales de tu proyecto.

Las **convenciones de nombres** (PascalCase, prefijos `Fn / Sp / Async` , etc.) son recomendaciones de equipo: adóptalas para mantener consistencia, pero la plantilla funciona con cualquier convención que elijas.

1. Qué generan los scripts

Script	Resultado
<code>New-MvcProject.ps1 -Name MiApp</code>	Proyecto MVC completo y autónomo: <code>BaseController</code> (sesión + <code>ModoDesarrollo</code>) , <code>HomeController</code> , vistas, <code>Web.config</code> , <code>Global.asax</code> , <code>App_Start</code> , bundles. Su propia <code>.sln</code> .
<code>Add-MvcModule.ps1 -Name X -SolutionPath .\MiApp</code>	Segundo proyecto MVC que referencia al principal (<code>ProjectReference</code>), hereda su <code>BaseController</code> , hereda la cadena de conexión, corre en su propio puerto y queda registrado en la <code>.sln</code> .

Modelo de enlace: el módulo hace `using MiApp.Controllers;` y su `HomeController : BaseController` . Así reutiliza la lógica de sesión/seguridad del principal.

2. Estructura del proyecto

MiApp/ ├ App_Start/ ├── RouteConfig.cs ← rutas (URL → controlador/acción) ├── FilterConfig.cs ← filtros globales (manejo de errores) └ BundleConfig.cs ← agrupado de CSS/JS ├ Controllers/ ├── BaseController.cs ← controlador base (sesión / ModoDesarrollo) └ HomeController.cs ← ejemplo, hereda de BaseController ├ Models/ └ Views/ ├── _ViewStart.cshtml ← define el layout por defecto ├── Shared/_Layout.cshtml ← plantilla común + vistas reutilizables (partials) └ Home/Index.cshtml ← vista de la acción Home/Index ├ Content/ ├ Scripts/ ├ Global.asax(.cs) ← arranque (registra rutas/bundles) └ Web.config ← conexión SQL, ModoDesarrollo, bindings	
---	--

Regla mental: una *página* = una **acción** en un *controller* + una **vista** `.cshtml` con el mismo nombre, dentro de la carpeta del controller.

2.1 ¿Dónde pongo cada cosa? ¿Necesito otro proyecto o script?

La pregunta clave es: **¿esto lo usa una sola app, o lo comparten varias?**

Quiero agregar...	¿Dónde va?	¿Otro proyecto/script?
Estilos (CSS) de una página	<code>Content/site.css</code> (o un <code>.css</code> nuevo) + regístralo en <code>BundleConfig.cs</code>	No
JavaScript de una página/módulo	<code>Scripts/main.js</code>	No
JS/CSS global (helpers, alertas, baseUrl)	<code>Scripts/config.js</code> + <code>Content/</code> y referencias en <code>_Layout.cshtml</code>	No

Componente visual reutilizable (tarjeta, tabla, menú) ¿quiero agregar...	¿Dónde va? partial en Views/Shared/_NombreComponente.cshtml	No ¿Otro proyecto/script?
Imágenes / fuentes / librerías de terceros	Content/ (imágenes/css) y Scripts/ (libs)	No
Una página nueva (pantalla)	un controller + su carpeta de vistas (ver §4)	No
Una funcionalidad grande e independiente (otra app que se publica aparte)	un módulo	Sí → Add-MvcModule.ps1
Lógica/seguridad compartida por TODAS las apps	BaseController del proyecto principal (los módulos lo heredan)	No (ya lo da New-MvcProject.ps1)
Un sistema nuevo desde cero	proyecto nuevo	Sí → New-MvcProject.ps1

Regla práctica:

- Casi todo lo nuevo (páginas, componentes, estilos, JS, tablas, SPs) NO necesita otro proyecto ni script. Se agrega dentro del proyecto actual en las carpetas de arriba.
- Solo creas otro proyecto con Add-MvcModule.ps1 cuando la nueva parte debe ser una aplicación separada (se publica/despliega aparte, tiene su propio puerto/URL), pero quieres que reutilice la sesión y el BaseController del principal.
- Solo usas New-MvcProject.ps1 cuando arrancas un sistema nuevo sin relación con el actual.

¿Y los componentes "compartidos entre proyectos"? Un partial en Views/Shared/ solo lo ve su proyecto. Si necesitas que un módulo reutilice un componente del principal, las opciones son:

1. Heredarlo vía BaseController (lógica/datos: ChildActions definidos en el principal).
2. Copiar el partial/CSS/JS al módulo (lo más simple para vistas).
3. Para algo realmente común a muchos módulos, ponlo en el proyecto principal y haz que los módulos lo consuman a través de la referencia que ya crea Add-MvcModule.ps1.

3. Convenciones de nombres (recomendadas)

Para que cualquier miembro del equipo lea el código sin contexto, conviene usar **PascalCase en todas las capas** (BD, C#, vistas) y prefijos consistentes:

Objeto	Prefijo / patrón	Ejemplo
Tabla	Entidad / EntidadRelacion	Producto , ProductoCategoria
Columna FK	Ref...	RefCategoriaId
Función SQL (lee datos)	Fn...	FnProductoTodos
Procedimiento (escribe datos)	Sp...	SpProductoGuardar
Método controlador AJAX	Async...	AsyncGuardar

Convenciones de tabla recomendadas: primera columna `Id INT` PK auto-increment, y columnas de auditoría `IdUsuario INT` + `FechaRegistro DATETIME` en cada tabla.

Nada de esto es obligatorio para que el proyecto compile — es una convención de equipo. Si tu proyecto ya usa otra, mantén la suya. Lo importante es ser consistente.

4. Crear una página nueva (controller → endpoint → vista)

4.1 Controller (hereda de BaseController)

```

using System.Web.Mvc;

namespace MiApp.Controllers
{
    public class ProductoController : BaseController
    {
        // GET: /Producto ó /Producto/Index
        public ActionResult Index()
        {
            if (!ValidaSesion()) return Salir(); // patrón estándar de la plantilla

            ViewBag.Title = "Productos";
            return View();
        }

        // GET: /Producto/Detalle/5
        public ActionResult Detalle(int id)
        {
            if (!ValidaSesion()) return Salir();
            // ... cargar datos por id ...
            return View();
        }
    }
}

```

4.2 El endpoint (ruta) ya existe

La ruta por defecto en `App_Start/RouteConfig.cs` es `{controller}/{action}/{id}`, así que `/Producto/Detalle/5` funciona sin configurar nada. Solo agrega rutas si necesitas URLs especiales:

```

routes.MapRoute(
    name: "ProductoPorCodigo",
    url: "p/{codigo}",
    defaults: new { controller = "Producto", action = "PorCodigo" }
);

```

4.3 La vista

Crea `Views/Producto/Index.cshtml` (carpeta = nombre del controller sin "Controller"):

```

@{ ViewBag.Title = "Productos"; }
<h1>Productos</h1>
<table class="table">
    <thead><tr><th>Id</th><th>Nombre</th></tr></thead>
    <tbody>@* recorres tu modelo aquí *@</tbody>
</table>

```

`_ViewStart.cshtml` ya aplica `Views/Shared/_Layout.cshtml`; solo escribes el contenido.

4.4 Tipos de endpoint comunes

```
// Página HTML
public ActionResult Index() { return View(); }

// Recibe formulario (POST)
[HttpPost]
public ActionResult Guardar(ProductoVm modelo)
{
    if (!ValidaSesion()) return Salir();
    if (!ModelState.IsValid) return View(modelo);
    // ... guardar ...
    return RedirectToAction("Index");
}

// Endpoint JSON para AJAX
public JsonResult Listar()
{
    if (!ValidaSesion()) return Json(new { ok = false }, JsonRequestBehavior.AllowGet);
    var datos = new[] { new { id = 1, nombre = "A" } };
    return Json(datos, JsonRequestBehavior.AllowGet);
}
```

4.5 Métodos AJAX con contrato JSON uniforme (recomendado)

Para llamadas asíncronas conviene un único formato de respuesta en TODO el proyecto:

```
{ estado: bool, mensaje: string, data: object }
```

```
// POST: /Producto/AsyncGuardar
[HttpPost]
public JsonResult AsyncGuardar(string nombre, decimal precio)
{
    if (!ValidaSesion())
        return Json(new { estado = false, mensaje = "Sesión inválida.", data = (object)null });

    // ... lógica / guardar ...
    bool ok = true;

    return Json(new
    {
        estado = ok,
        mensaje = ok ? "Guardado correctamente." : "No se pudo guardar.",
        data = ok ? new { id = 123 } : null
    });
}
```

El cliente siempre lee `estado` para decidir, `mensaje` para mostrar y `data` para los datos.

5. Diseño y componentes (vistas reutilizables)

5.1 Layout común

`Views/Shared/_Layout.cshtml` es el marco de todas las páginas (navbar, footer, CSS/JS). Incluye CSS/JS propios con **bundles** (`App_Start/BundleConfig.cs`):

```
bundles.Add(new StyleBundle("~/Content/css").Include("~/Content/site.css"));
bundles.Add(new ScriptBundle("~/bundles/app").Include("~/Scripts/main.js"));
```

y en el layout: `@Styles.Render("~/Content/css") / @Scripts.Render("~/bundles/app")` .

5.2 Vistas parciales (componentes)

Trozos reutilizables (tarjeta, tabla, menú) van en `Views/Shared/` con guion bajo:

```
@* Views/Shared/_MenuLateral.cshtml *@
<ul class="nav flex-column">
    <li><a href="@Url.Action("Index", "Home")">Inicio</a></li>
    <li><a href="@Url.Action("Index", "Producto")">Productos</a></li>
</ul>
```

Se incluyen con `@Html.Partial("_MenuLateral")`. Si el componente debe consultar datos por su cuenta, usa un **child action**:

```
[ChildActionOnly]
public ActionResult MenuModulos()
{
    var modulos = /* consulta a BD */;
    return PartialView("_MenuModulos", modulos);
}
```

```
@Html.Action("MenuModulos", "Home")    @* en el layout *@
```

6. Organización del JavaScript (recomendado)

Mantén dos archivos en `Scripts/` para separar lo global de lo específico:

Archivo	Contenido
<code>config.js</code> (o <code>core.js</code>)	Variables de entorno públicas + objetos/utilidades reutilizables en todo el proyecto (alertas, helpers AJAX, baseUrl).
<code>main.js</code>	Funciones específicas del módulo/página actual.

Cada vista solo **instancia** objetos ya definidos en `main.js` y administra eventos:

```
// main.js
var Producto = {
    guardar: function (datos) {
        $.post(app.baseUrl + "Producto/AsyncGuardar", datos, function (r) {
            if (r.estado) app.alerta.exito(r.mensaje); // helper global (config.js)
            else app.alerta.error(r.mensaje);
        });
    }
};
```

```
@* Views/Producto/Index.cshtml *@
<button id="btnGuardar">Guardar</button>
<script>
    $("#btnGuardar").on("click", function () {
        Producto.guardar({ nombre: $("#nombre").val(), precio: $("#precio").val() });
    });
</script>
```

7. Modelo de datos con Entity Framework (Database-First)

Primero existe la base de datos; EF genera las clases desde un `.edmx`.

7.1 Cadena de conexión con metadatos

Al agregar un `.edmx`, EF crea una conexión EntityClient (con metadatos):

```
<add name="MiAppEntities"
  connectionString="metadata=res://*/Models.Modelo.csdl|res://*/Models.Modelo.ssdl|res://*/Models.Modelo.msl;
  provider=System.Data.SqlClient;
  provider connection string=&quot;data source=SERVIDOR;initial catalog=MIBD;
  user id=USUARIO;password=CLAVE;multipleactiveresultsets=True;trustservercertificate=True&quot;;"
  providerName="System.Data.EntityClient" />
```

7.2 Generar el modelo en Visual Studio

1. Clic derecho en **Models** → **Add** → **New Item** → **ADO.NET Entity Data Model**.
2. **EF Designer from database** (Database-First).
3. Selecciona la conexión a tu SQL Server.
4. Marca **Tablas, Vistas, funciones y procedimientos** que necesites.
5. Finish → genera el `.edmx`, el `DbContext` (`MiAppEntities`) y las clases (`Fn*_Result`, `Sp*_Result`).

Cada cambio en la BD: abre el `.edmx` → **Update Model from Database** para regenerar.

7.3 Usar el modelo desde un controller

```
public class ProductoController : BaseController
{
    private MiAppEntities db = new MiAppEntities();

    public ActionResult Index()
    {
        if (!ValidaSesion()) return Salir();

        var lista = db.Productos.Where(p => p.Estado == 1).ToList(); // tabla directa
        var porFn = db.FnProductoTodos().ToList(); // función (List<FnProductoTodos_Result>)
        return View(lista);
    }

    protected override void Dispose(bool disposing)
    {
        if (disposing) db.Dispose();
        base.Dispose(disposing);
    }
}
```

8. Stored Procedures (patrón de salida uniforme)

Conviene que todo SP de escritura devuelva un **código + mensaje** consistentes:

- Salida Respuesta INT → **1** éxito · **0** validación/incorrecto · **-1** excepción.
- Salida Mensaje VARCHAR(MAX) → texto de confirmación o error.
- (Opcional) entrada `IdUsuario` INT para auditar quién ejecuta.

```

CREATE PROCEDURE SpProductoGuardar
    @Nombre      VARCHAR(100),
    @Precio      DECIMAL(18,2),
    @IdUsuario   INT,
    @Respuesta   INT          OUTPUT,
    @Mensaje     VARCHAR(MAX) OUTPUT
AS
BEGIN
    SET NOCOUNT ON;
    BEGIN TRY
        IF (@Nombre IS NULL OR LEN(@Nombre) = 0)
            BEGIN
                SET @Respuesta = 0; SET @Mensaje = 'El nombre es obligatorio.'; RETURN;
            END

        INSERT INTO Producto (Nombre, Precio, Estado, IdUsuario, FechaRegistro)
        VALUES (@Nombre, @Precio, 1, @IdUsuario, GETDATE());

        SET @Respuesta = 1; SET @Mensaje = 'Guardado correctamente.';
    END TRY
    BEGIN CATCH
        SET @Respuesta = -1; SET @Mensaje = ERROR_MESSAGE();
    END CATCH
END

```

8.1 Importar el SP al modelo EF

1. Abre el `.edmx` → **Update Model from Database**.
2. Pestaña **Stored Procedures and Functions** → marca tu SP → Finish.
3. Si devuelve filas: **Model Browser** → **Function Imports** → "Returns a Collection of" para que genere la clase `Sp..._Result`.

8.2 Llamar el SP con parámetros de salida

```

using System.Data.Entity.Core.Objects;    // ObjectParameter

[HttpPost]
public JsonResult AsyncGuardar(string nombre, decimal precio)
{
    if (!ValidaSesion())
        return Json(new { estado = false, mensaje = "Sesión inválida.", data = (object)null });

    var respuesta = new ObjectParameter("Respuesta", typeof(int));
    var mensaje   = new ObjectParameter("Mensaje",   typeof(string));

    db.SpProductoGuardar(nombre, precio, UsuarioId, respuesta, mensaje);

    int codigo = Convert.ToInt32(respuesta.Value);    // 1 / 0 / -1
    return Json(new
    {
        estado = (codigo == 1),
        mensaje = Convert.ToString(mensaje.Value),
        data    = (codigo == 1) ? new { } : null
    });
}

```

SP que devuelve filas:

```
var productos = db.SpProductoPorCategoria(categoriaId).ToList(); // List<SpProductoPorCategoria_Result>
```

9. Sesión / autenticación con JWT

9.1 Idea general

- `BaseController.ValidaSesion(int refPermisoId = 0)` es el punto único de control.
- Con `ModoDesarrollo = true` en `Web.config`, **siempre devuelve true** (probar sin login).
- Login real (token + BD): un método `CreaSesion(...)` genera un **JWT firmado**, lo guarda en BD y lo deja en cookies (`Sesion` / `Authorization`).
- `ValidaSesion()` lee la cookie/token, **valida la firma del JWT** y confirma la sesión en BD.
- Si varios proyectos comparten dominio + BD, escribe las cookies **sin Domain/Path** (SSO).
- Cambia `ModoDesarrollo` a `false` cuando el login real esté listo.

9.2 Proteger los endpoints (que no queden expuestos)

El problema: cualquier acción es accesible por URL si no se valida. Hay dos capas de defensa, úsalas juntas:

a) **En cada acción** (rápido, ya lo hace la plantilla):

```
public ActionResult Index()
{
    if (!ValidaSesion()) return Salir();    // sin token válido → fuera
    // ...
}
```

b) **Un filtro/atributo JWT** para no repetir la validación y blindar TODO el controller. Crea un atributo que valide el JWT antes de ejecutar la acción:

```
using System;
using System.IdentityModel.Tokens.Jwt;
using System.Text;
using System.Web.Mvc;
using Microsoft.IdentityModel.Tokens;

public class JwtAuthorizeAttribute : ActionFilterAttribute
{
    public override void OnActionExecuting(ActionExecutingContext ctx)
    {
        // En desarrollo se omite (igual que ModoDesarrollo)
        if (Seguridad.ModoDesarrollo) return;

        var token = ctx.HttpContext.Request.Cookies["Authorization"]?.Value;
        if (string.IsNullOrEmpty(token) || !Seguridad.ValidarJwt(token))
        {
            // AJAX → 401 JSON; navegador → redirige al login
            if (ctx.HttpContext.Request.IsAjaxRequest())
                ctx.Result = new HttpStatusCodeResult(401, "No autorizado");
            else
                ctx.Result = new RedirectResult("~/Home/Login");
        }
    }
}
```

Y lo aplicas a nivel de **controller** (protege todas sus acciones de un golpe):

```
[JwtAuthorize]
public class ProductoController : BaseController { /* ... */ }
```

Validación/firma del token (centralizada en una clase `Seguridad`):


```

public static class Seguridad
{
    // La KEY se LEE de configuración, nunca está escrita en el código:
    private static string JwtKey => System.Configuration.ConfigurationManager.AppSettings["JwtKey"];
    public static bool ModoDesarrollo =>
        System.Configuration.ConfigurationManager.AppSettings["ModoDesarrollo"] == "true";

    public static string CrearJwt(int usuarioId, string nombre)
    {
        var key = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(JwtKey));
        var creds = new SigningCredentials(key, SecurityAlgorithms.HmacSha256);
        var token = new JwtSecurityToken(
            issuer: "MiApp", audience: "MiApp",
            claims: new[] { new System.Security.Claims.Claim("uid", usuarioId.ToString()) },
            expires: DateTime.UtcNow.AddHours(8), signingCredentials: creds);
        return new JwtSecurityTokenHandler().WriteToken(token);
    }

    public static bool ValidarJwt(string token)
    {
        try
        {
            new JwtSecurityTokenHandler().ValidateToken(token, new TokenValidationParameters
            {
                ValidIssuer = "MiApp", ValidAudience = "MiApp",
                IssuerSigningKey = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(JwtKey)),
                ValidateIssuerSigningKey = true, ValidateLifetime = true,
                ClockSkew = TimeSpan.Zero
            }, out _);
            return true;
        }
        catch { return false; } // firma inválida, expirado o manipulado → false
    }
}

```

Paquetes NuGet: `System.IdentityModel.Tokens.Jwt` y `Microsoft.IdentityModel.Tokens`.

9.3 La KEY y los secretos NO se publican (appSettings externo)

La clave de firma del JWT, contraseñas de BD, API keys de SMS/correo, etc. **nunca** van escritas en el código ni se suben al repositorio ni al servidor. Se leen de configuración, y la configuración con secretos se mantiene **fuera del control de versiones**.

Patrón recomendado — appSettings en archivo externo con configSource:

`Web.config` (esto SÍ se versiona; no contiene secretos, solo apunta al archivo):

```
<appSettings configSource="appSettings.secret.config" />
```

`appSettings.secret.config` (esto NO se versiona ni se publica — va en `.gitignore`):

```

<appSettings>
  <add key="ModoDesarrollo" value="false" />
  <add key="JwtKey" value="CLAVE-LARGA-Y-ALEATORIA-MIN-32-CARACTERES" />
  <add key="SmsApiKey" value="..." />
</appSettings>

```

Alternativa equivalente con el atributo `file` (mezcla con los del `Web.config`):

```

<appSettings file="appSettings.secret.config">
  <add key="webpages:Enabled" value="false" />
</appSettings>

```

.gitignore (imprescindible):

```
appSettings.secret.config
*.secret.config
connectionStrings.secret.config
```

Buenas prácticas:

- Versiona una **plantilla** `appSettings.secret.config.example` (con valores ficticios) para que cada desarrollador la copie y rellene en local.
- En el servidor, coloca el archivo de secretos **una sola vez** y no lo sobrescribas al publicar (en *Publish* de Visual Studio, marca el archivo como *"Do not copy / Exclude"* o usa **Web.config Transforms** `Web.Release.config` para inyectar los valores de producción).
- La `JwtKey` debe ser larga y aleatoria (≥ 32 caracteres). Si se filtra, **rótala** (cámbiala): todos los tokens firmados con la anterior quedan inválidos automáticamente.
- Lo mismo aplica a la `connectionString`: puedes externalizarla con `<connectionStrings configSource="connectionStrings.secret.config" />`.

⚠ Si una key ya se subió alguna vez al repo, considérala comprometida: cámbiala. Quitarla en un commit posterior no la borra del historial de git.

10. Agregar un módulo (proyecto aparte enlazado)

Cuando una parte del sistema deba ser una app independiente, créala como **módulo**:

```
.\Add-MvcModule.ps1 -Name Inventario -SolutionPath .\MiApp
```

Esto genera un proyecto MVC completo que **referencia al principal** y cuyo `HomeController` **hereda del `BaseController`** del principal (reutiliza sesión/seguridad).

- **Desarrollo**: cada módulo corre en su propio puerto IIS Express (F5 sobre el módulo).
- **Producción**: se publican todos bajo **un mismo sitio IIS** como *Applications* (`/`, `/Inventario`, ...). Al compartir dominio + BD comparten la sesión (SSO).

Si tu proyecto principal mantiene un catálogo de módulos (p. ej. una tabla `Modulos` con nombre, icono, color, estado), registra ahí el nuevo módulo para que el menú lo liste.

11. Flujo completo de extremo a extremo (ejemplo real)

Vamos a crear una pantalla **"Productos"** que **lista** desde la BD y **guarda** uno nuevo. Esto muestra cómo se conecta todo: BD → EF → controller → endpoint → vista → JS → BD.

```
[ Vista .cshtml ] --(click)--> [ main.js ] --(AJAX)--> [ Controller (endpoint) ]
      ^                               |
      |                               v
render HTML <--- ViewBag/Model --- [ Controller ] <--- [ EF: db.Fn / db.Sp ] <---> [ SQL Server ]
```

Paso 1 — Base de datos (tabla + función + SP)

```

-- Tabla
CREATE TABLE Producto (
    Id            INT IDENTITY(1,1) PRIMARY KEY,
    Nombre        VARCHAR(100) NOT NULL,
    Precio        DECIMAL(18,2) NOT NULL,
    Estado        INT NOT NULL DEFAULT 1,
    IdUsuario     INT NOT NULL,
    FechaRegistro DATETIME NOT NULL DEFAULT GETDATE()
);
GO
-- Función para LEER
CREATE FUNCTION FnProductoTodos () RETURNS TABLE AS
RETURN SELECT Id, Nombre, Precio FROM Producto WHERE Estado = 1;
GO
-- SP para ESCRIBIR (patrón Respuesta/Mensaje)
CREATE PROCEDURE SpProductoGuardar
    @Nombre VARCHAR(100), @Precio DECIMAL(18,2), @IdUsuario INT,
    @Respuesta INT OUTPUT, @Mensaje VARCHAR(MAX) OUTPUT
AS
BEGIN
    SET NOCOUNT ON;
    BEGIN TRY
        INSERT INTO Producto (Nombre, Precio, Estado, IdUsuario, FechaRegistro)
        VALUES (@Nombre, @Precio, 1, @IdUsuario, GETDATE());
        SET @Respuesta = 1; SET @Mensaje = 'Guardado correctamente.';
    END TRY
    BEGIN CATCH
        SET @Respuesta = -1; SET @Mensaje = ERROR_MESSAGE();
    END CATCH
END

```

Paso 2 — Modelo EF

Abre el `.edmx` → **Update Model from Database** → marca la tabla `Producto`, la función `FnProductoTodos` y el SP `SpProductoGuardar`. EF genera `Producto`, `FnProductoTodos_Result` y el método `SpProductoGuardar(...)` en el `DbContext`.

Paso 3 — Controller con sus endpoints

`Controllers/ProductoController.cs`:

```

using System;
using System.Linq;
using System.Web.Mvc;
using System.Data.Entity.Core.Objects;
using MiApp.Models;           // namespace del .edmx

namespace MiApp.Controllers
{
    public class ProductoController : BaseController
    {
        private MiAppEntities db = new MiAppEntities();

        // 1) Endpoint de PÁGINA: GET /Producto -> devuelve la vista con datos
        public ActionResult Index()
        {
            if (!ValidaSesion()) return Salir();
            var productos = db.FnProductoTodos().ToList();    // LEER vía función
            return View(productos);                            // pasa el modelo a la vista
        }

        // 2) Endpoint AJAX: POST /Producto/AsyncGuardar -> devuelve JSON uniforme
        [HttpPost]
        public JsonResult AsyncGuardar(string nombre, decimal precio)
        {
            if (!ValidaSesion())
                return Json(new { estado = false, mensaje = "Sesión inválida.", data = (object)null });

            var respuesta = new ObjectParameter("Respuesta", typeof(int));
            var mensaje = new ObjectParameter("Mensaje", typeof(string));

            db.SpProductoGuardar(nombre, precio, UsuarioId, respuesta, mensaje); // ESCRIBIR vía SP

            int codigo = Convert.ToInt32(respuesta.Value);    // 1 / 0 / -1
            return Json(new
            {
                estado = (codigo == 1),
                mensaje = Convert.ToString(mensaje.Value),
                data = (codigo == 1) ? new { } : null
            });
        }

        protected override void Dispose(bool disposing)
        {
            if (disposing) db.Dispose();
            base.Dispose(disposing);
        }
    }
}

```

Endpoints disponibles automáticamente (ruta {controller}/{action}/{id}): GET /Producto y POST /Producto/AsyncGuardar . No hay que tocar RouteConfig.cs .

Paso 4 — Vista

Views/Producto/Index.cshtml (recibe List<FnProductoTodos_Result> como modelo):

```
@model IEnumerable<MiApp.Models.FnProductoTodos_Result>
@{ ViewBag.Title = "Productos"; }

<h1>Productos</h1>

<input id="nombre" placeholder="Nombre" />
<input id="precio" placeholder="Precio" type="number" />
<button id="btnGuardar" class="btn btn-primary">Guardar</button>

<table class="table" id="tabla">
    <thead><tr><th>Id</th><th>Nombre</th><th>Precio</th></tr></thead>
    <tbody>
        @foreach (var p in Model)
        {
            <tr><td>@p.Id</td><td>@p.Nombre</td><td>@p.Precio</td></tr>
        }
    </tbody>
</table>
```

Paso 5 — JavaScript del módulo

Scripts/main.js (regístralo en `BundleConfig.cs` y renderiza el bundle en `_Layout`):

```
$(function () {
    $("#btnGuardar").on("click", function () {
        $.post("/Producto/AsyncGuardar",
            { nombre: $("#nombre").val(), precio: $("#precio").val() },
            function (r) {
                if (r.estado) { // contrato { estado, mensaje, data }
                    alert(r.mensaje);
                    location.reload(); // recargar la tabla
                } else {
                    alert("Error: " + r.mensaje);
                }
            });
    });
});
```

Paso 6 — Menú y prueba

Agrega el enlace en `Views/Shared/_Layout.cshtml` (o en tu partial de menú):

```
<li><a href="@Url.Action("Index", "Producto")">Productos</a></li>
```

Ejecuta con `ModoDesarrollo = true`, entra a `/Producto`, guarda uno y verifica que aparece. **Listo: ese es el ciclo completo** que repetirás para cualquier pantalla nueva.

12. Checklist paso a paso para crear algo nuevo

A. Base de datos

- ☐ Crea la **tabla** (Id INT PK auto-increment + IdUsuario + FechaRegistro; FKs con Ref).
- ☐ Crea **función** `Fn...` para leer (SELECT) y **SP** `Sp...` para escribir (con `@Respuesta` / `@Mensaje`).

B. Modelo de datos 3. [] Abre el `.edmx` → **Update Model from Database** → marca tabla, función y SP. 4. [] Si el SP devuelve filas, confírmalo en **Model Browser** → **Function Imports**.

C. Backend (controller + endpoints) 5. [] Crea `Controllers/<Entidad>Controller.cs` que **hereda de `BaseController`**. 6. [] **Protege el controller:** `[JwtAuthorize]` y/o `if (!ValidaSesion()) return Salir();` en cada acción. 7. [] Acción de **página** (`Index`) que devuelve `View(datos)`. 8. [] Acciones **AJAX** (`Async...`) que devuelven JSON `{ estado, mensaje, data }`. 9. [] Conecta la BD: `db.Fn...` para leer, `db.Sp...` para escribir. 10. [] (Opcional) ruta personalizada en `RouteConfig.cs` solo si necesitas una URL especial.

D. Frontend (vista + diseño + JS) 11. [] Crea la **vista** en `Views/<Entidad>/Index.cshtml` con `@model ...`. 12. [] Extrae lo reutilizable a **partials** en `Views/Shared/_Componente.cshtml`. 13. [] Estilos en `Content/` y JS en `Scripts/main.js`; **regístralos en `BundleConfig.cs`**. 14. [] El JS hace el AJAX (mandando la cookie/token) y lee `estado / mensaje / data`.

E. Seguridad y secretos 15. ☐ La `JwtKey` y demás secretos van en `appSettings.secret.config` (externo), **no** en el código. 16. ☐ Ese archivo está en `.gitignore` y excluido del *Publish* (no se sube al repo ni al servidor).

F. Integración y prueba 17. ☐ Enlaza la página en el menú (`_Layout` o `partial`). 18. ☐ Prueba con `ModoDesarrollo = true`; luego valida con JWT/sesión real (`ModoDesarrollo = false`). 19. ☐ ¿Es una app independiente? → conviértela en **módulo** con `Add-MvcModule.ps1`.

13. Referencia rápida de comandos

```
# Crear un proyecto/portal nuevo
.\New-MvcProject.ps1 -Name MiApp -SqlServer "localhost\SQLEXPRESS" -Database "MiBD"

# Agregar un módulo a una solución existente
.\Add-MvcModule.ps1 -Name Inventario -SolutionPath .\MiApp

# Compilar desde línea de comandos (MSBuild de VS 2022)
& "C:\Program Files\Microsoft Visual Studio\2022\Community\MSBuild\Current\Bin\MSBuild.exe" .\MiApp\MiApp.sln /t:Build
```